DOCUMENT 18

Analysis Guide

VLM analysis pipeline, shoot-match API, AnalysisResult structure, Lab Workb

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

About This Guide

The NGW analysis system transforms a reference photograph — or a set of natural-language inputs — into a fully structured lighting prescription. Every surface in the system traces back to a single Python call: `analyze_image()` in `engine/orchestrator.py`. This guide documents the seven-stage pipeline, every field in `AnalysisResult`, the shoot-match and lab API endpoints, the results screen card inventory, debug overlay generation, and edge-case handling — drawn directly from the source.

7-Stage	4	24	29
VLM analysis pipeline	Active classifiers	Visual cue signals	Lab API endpoints

AUTH MODEL

- POST /shoot-match — user-facing; requires a valid session JWT only.
- POST /lab/analyze — dev only; JWT email must appear in NGW_DEV_EMAILS.
- GET /lab/reference-dataset/.../debug-overlay — dev only; same auth.
- Set NGW_DEV_MODE=1 locally to bypass auth. Never in production.

Surface	Who uses it	Auth
POST /shoot-match	All users	Session JWT
POST /upload-reference	All users	Session JWT
POST /lab/analyze	Curators / devs	JWT + NGW_DEV_EMAILS

Lab Workbench (UI)	Curators / devs	JWT + NGW_DEV_EMAILS
Debug overlay	Curators / devs	JWT + NGW_DEV_EMAILS

Section 1 — What Is Image Analysis?

Image analysis is the process of reading a reference photograph and determining exactly what lighting setup was used to make it. The NGW engine reads pixel-level evidence — shadow direction and softness, catchlight shape and topology, highlight axis symmetry, background fall-off, specular behavior, and more — and maps that evidence to one of 28 canonical lighting patterns from the reference dataset.

Analysis is triggered in two ways. When a user uploads a reference image, `POST /shoot-match` runs the full pipeline and returns a lighting prescription. When a curator uses the Lab Workbench, `POST /lab/analyze` runs the same pipeline with every internal structure exposed for inspection.

The pipeline is not a single model call. It is a deterministic orchestration of computer-vision passes, multi-stage VLM reconstruction, four independent classifiers, and a solver chain — all producing structured evidence that feeds a single pattern-resolution function. The authoritative pattern is always the output of `resolve_pattern_candidates()`, never a direct model response.

Analysis vs. Shoot-Match

The shoot-match endpoint handles two distinct input modes. When no reference image is provided, it matches natural-language inputs (subject, mood, environment, gear) to the closest pattern using heuristic scoring. When a reference image is present, the full analysis pipeline runs and the natural-language inputs are used only to refine gear recommendations.

Mode	Input	Pipeline depth
Text-only match	subject + mood + environment	Heuristic only — no VLM
Reference analysis	reference image (JPEG/PNG)	Full 7-stage pipeline
Lab analyze	image upload (multipart)	Full pipeline + debug overlay

Section 2 — The 7-Stage Analysis Pipeline

Every reference image passes through seven sequential stages inside `analyze_image()` in `engine/orchestrator.py`. Each stage reads from and writes to the shared `AnalysisResult` object. No stage produces the final pattern — all seven contribute evidence that `resolve_pattern_candidates()` weighs in Stage 6.

Stage	Name	Module	Writes to AnalysisResult
1	Signal Extraction	engine/image_analysis.py	description, cue_report (24 cues)
2	Vision Pipeline	engine/vision_pipeline.py	vision_data (face box, catchlights, regions)
3	Cue Inference	engine/cue_inference.py	cue_inference_result
4	VLM Reconstruction	engine/vlm_reconstruction.py	vlm_reconstruction (12 dimensions)
5	Solver Chain	engine/solver_chain.py	solver_result (contradictions resolved)
6	Pattern Resolution	engine/orchestrator.py	pattern_candidates, authoritative_pattern
7	Reference Read	engine/reference_read.py	reference_analysis (3-layer deep read)

Stage 1 — Signal Extraction

`describe_image()` runs the image through the full cue extraction pipeline and populates `AnalysisResult.cue_report` — a `VisualCueReport` with 24 named fields. Each field is an Optional dataclass that may be None if the signal could not be computed (e.g., face-dependent cues when no face is detected). `cue_report.cues_computed` records how many of the 24 were successfully extracted.

Stage 2 — Vision Pipeline

`_run_extended_pipeline()` executes 16+ independent signal passes against the image using computer vision (MediaPipe, OpenCV). Key outputs include: face bounding box, face detection confidence score, catchlight positions and topology, highlight axis map, shadow direction and hardness, region attribution percentages, and background illumination pattern. Results land in `AnalysisResult.vision_data`.

Stage 3 — Cue Inference

`run_cue_inference_pipeline()` interprets the raw cues from Stage 1 using a rule-based inference engine. It synthesizes shadow direction + height + hardness into a structured lighting hypothesis and feeds `AnalysisResult.cue_inference_result` which becomes the `cue_inference` classifier input in Stage 6.

Stage 4 — VLM Reconstruction

The VLM (Vision-Language Model) is called once and asked to reconstruct the lighting setup across 12 structured dimensions: direction, height, source size, modifier family,

light count, key role, fill role, rim role, background role, bounce likelihood, mixed sources, and pattern name. The response is parsed into `AnalysisResult.vlm_reconstruction`. The `pattern_name` dimension feeds the `lighting_inference` classifier in Stage 6.

Stage 5 — Solver Chain

Six solvers run in sequence to detect and resolve contradictions between the four classifier inputs: consensus_solver consistency_engine contradiction_engine lighting_simulator hypothesis_validator solver_trace. The solver result is stored in `AnalysisResult.solver_result` and influences the confidence adjustments applied in Stage 6.

Stage 6 — Pattern Resolution

`resolve_pattern_candidates()` is the authoritative ranking function. It collects candidate patterns from all four classifiers, applies priority ordering and any contradiction demotions, and returns a `PatternCandidates` object. The `primary` candidate becomes `authoritative_pattern` on `AnalysisResult`.

Stage 7 — Reference Read

`build_reference_photo_analysis()` performs a three-layer deep read of the image: (1) technical read — hardware and modifier identification, (2) aesthetic read — mood, contrast, color temperature, (3) recreation guide — step-by-step instructions for reproducing the setup. This populates `AnalysisResult.reference_analysis` and feeds the reference-image cards in the results screen.

Section 3 — AnalysisResult Structure

`AnalysisResult` is a Python dataclass defined in `engine/orchestrator.py` (lines 649–688). It uses `__slots__` for performance. All 24 fields are set to safe defaults in `__init__` — downstream consumers should always check `ok` before reading inference fields.

Field	Type	Description
ok	bool	False if pipeline raised an unrecoverable error
description	Dict[str, Any]	Raw image description from Stage 1 cue extraction
vision_data	Dict[str, Any]	Extended pipeline output — face box, catchlights, regions
classification	Dict[str, Any]	Image type classification (portrait / product / scene)
cue_report	VisualCueReport	24-field cue report; cues_computed tracks success count
cue_inference_result	Optional[Dict]	Stage 3 cue inference output (shadow pattern hypothesis)
lighting_intel	LightingIntelligence	Light count, modifier, pattern — pre-resolution summary
reference_analysis	ReferenceAnalysis	Three-layer deep read from Stage 7 (tech / aesthetic / guide)
pipeline_results	Optional[Dict]	Raw per-stage outputs; for debug inspection

vlm_description	Any	Raw VLM image description (Stage 1 model call)
vlm_reconstruction	Any	12-dimension structured reconstruction from Stage 4
solver_result	Any	Stage 5 contradiction analysis and resolution trace
debug_data	Dict[str, Any]	Diagnostic data — face_box, overlay path, timing
notes	List[str]	Human-readable warnings from any pipeline stage
authoritative_pattern	str	Final resolved pattern name (e.g. "rembrandt", "loop")
authoritative_pattern_source	str	Classifier that produced it (reference_read / lighting_inference / etc.)
pattern_candidates	PatternCandidates	Ranked candidates from all classifiers — primary + alternates
pattern_confidence	float	Primary candidate confidence score 0.0–1.0
pattern_confidence_label	str	strong (>0.75) / partial (≥0.50) / weak (<0.50)
face_validation	FaceValidation	Face detection quality — detected, confidence, quality tier, yaw, area ratio
signal_reliability	SignalReliability	Signal coverage — available/total, weak signals, missing signals
perception_explanation	PerceptionExplanation	Supporting/contradicting signals, ambiguity flags, reasoning text

edge_case_flags	EdgeCaseFlags	Boolean flags for blown highlights, mixed CT, B&W;, no-face, low-key

PatternCandidates

`PatternCandidates` is the core output of Stage 6. Its `to_dict()` method produces the `patternCandidates` key in every API response.

Field	Type	Description
primary	PatternCandidate	Highest-ranked candidate — defines authoritative_pattern
alternates	List[PatternCandidate]	Other viable candidates in confidence order
needs_review	bool	True when classifiers significantly disagree
contradictions	List[str]	Human-readable contradiction descriptions

Each `PatternCandidate` has: `pattern` (canonical name), `source` (classifier), `confidence` (0–1), `rank` (1-based within classifier output).

Section 4 — Pattern Resolution & Classifiers

`resolve_pattern_candidates()` collects candidates from four independent classifiers and applies a strict priority ordering. Only one function determines the authoritative pattern — downstream code must read `pattern_candidates.authoritative_pattern`, never reach directly into classifier outputs.

Classifier Priority Order

Priority	Classifier	Source module	Basis
0 (highest)	reference_read	engine/reference_read.py	Richest analysis — full three-layer deep read with VLM
1	lighting_inference	engine/lighting_inference.py	Vision-based catchlight topology + shadow geometry
2	cue_inference	engine/cue_inference.py	Shadow direction + height + hardness rule synthesis
3	light_structure	engine/vision_pipeline.py	Nose-shadow geometry direct pattern derivation
—	unknown fallback	orchestrator.py	When no classifier produces a result

Contradiction Demotion

When vision signals (triangle_isolation, shadow_density, lr_asymmetry, highlight_symmetry) strongly contradict the `reference_read` primary candidate, its effective priority is demoted from 0 to 2 and its confidence is halved. This allows

competing candidates that align better with raw pixel evidence to win. The demotion reason is recorded in `PatternCandidates.contradictions`.

Confidence Label Tiers

Label	Confidence range	Meaning
strong	> 0.75	Classifiers agree, signals clear — high trust
partial	0.50 – 0.75	Reasonable evidence — use with normal care
weak	< 0.50	Ambiguous signals — flag for curator review

28 CANONICAL PATTERNS

- split, rembrandt, loop, butterfly, clamshell, triangle, broad, short
- gobo, flat, rim_only, high_key, low_key, flat_fashion, window_portrait
- golden_hour, overcast_natural, ring_light, bare_bulb_editorial, strip_dramatic
- short_fashion_key, soft_editorial_key, editorial_rim_key
- tabletop_soft_product, bottle_backlight, athletic_rim_sculpt
- window_negative_fill, hybrid, unknown

Section 5 — Shoot-Match API

The shoot-match endpoint is the primary user-facing analysis surface. It accepts either natural-language inputs or a base64-encoded reference image. When an image is present the full 7-stage pipeline runs; otherwise heuristic matching is used.

POST /shoot-match

Request body (JSON)

```
{
  "subject":        "headshot",          // woman | man | child | couple | group …
  "mood":           "editorial",
  "environment":    "studio",             // studio | indoor | outdoor
  "ceiling":        "normal",             // normal | low
  "gearMode":       "anyGear",            // anyGear | myGear
  "gear":           ["softbox", "rim"],   // optional gear list
  "skinTone":       "medium",             // optional
  "referenceImage": "<base64-string>",    // optional — triggers full pipeline
  "masterMode":     null                  // optional override
}
```

Response (condensed)


```
{
  "status": "ok",
  "requestId": "abc123",
  "processingMs": 2340,
  "authoritative_pattern": "rembrandt",
  "patternCandidates": {
    "primary_candidate": { "pattern": "rembrandt", "source": "reference_read",
                           "confidence": 0.87, "confidence_label": "strong" },
    "alternate_candidates": [ { "pattern": "loop", "source": "lighting_inference",
                                "confidence": 0.61 } ],
    "needs_review": false,
    "contradictions": []
  },
  "faceValidation": { "face_detected": true, "face_quality": "good",
                      "face_confidence": 0.94, "face_yaw": 12.3,
                      "face_box_area_ratio": 0.18 },
  "signalReliability": { "signals_available": 19, "signals_total": 24,
                         "overall_signal_strength": 0.81,
                         "weak_signals": [], "missing_signals": ["pose_induced_sha…
  "edgeCaseFlags": { "no_face": false, "blown_highlights": false,
                     "mixed_color_temperature": false, "bw_processing": false },
  "lightingIntelligence": { ... },
  "referenceImageAnalysis": { ... },
  "cards": [ ... ]
}
```


POST /upload-reference

Accepts a multipart image upload. Runs the full analysis pipeline and returns the same response shape as `POST /shoot-match` with a reference image. Use this when the reference image is too large for base64 inline encoding (max inline size ≈ 8 MB; upload limit is 10 MB).

Section 6 — Lab Workbench

The Lab Workbench is the curator tool for running full-fidelity analysis with all internal structures exposed. It lives in the Workbench tab of the Lab screen (`ui/src/screens/LabScreen.jsx`) and is backed by `POST /lab/analyze`.

POST /lab/analyze

Request (multipart/form-data)

```
image:   <file upload>      # JPEG or PNG, max 10 MB
debug:   true | false       # query param — generates visual overlay
```

Response fields

```
{
  "status":         "ok",
  "image_path":     "/static/uploads/abc123.jpg",
  "description":    { ... },          // Stage 1 raw cue extraction
  "reference_analysis": { ... },      // Stage 7 three-layer deep read
  "vlm":            { ... },          // Stage 1 VLM description
  "vlm_available":  true,
  "vlm_reconstruction": {             // Stage 4 — 12 dimensions
    "direction":    "45_camera_left",
    "height":       "above_eye",
    "source_size":  "large_soft",
    "modifier_family": "softbox",
    "light_count":  2,
    "key_role":     "main_key",
    "fill_role":    "fill",
    "rim_role":     "none",
    "background_role": "none",
    "bounce_likelihood": "low",
    "mixed_sources": false,
    "pattern_name": "rembrandt"
  },
  "cv":             { ... },          // Stage 2 vision_data
  "classification": { ... },          // image type classification
  "lighting_inference": { ... },      // lighting_intel summary
  "solver":         { ... },          // Stage 5 solver chain result
  "analyzed_by":    "dev@example.com",
  "analyzed_at":    "2025-11-14T09:23:11Z",
  "debug_overlay_url": "/static/debug/abc123_overlay.jpg"  // only when debug=true
}
```

DEBUG OVERLAY FLAG

- Pass ?debug=true to generate a visual overlay image.
- The overlay annotates: face bounding box (cyan), shadow direction arrows (amber),
- catchlight positions (white dots), highlight regions (blue), background regions (green).
- URL is returned in debug_overlay_url — accessible at GET /static/debug/<filename>.
- Overlays are stored in /static/debug/ and not automatically cleaned up.

Workbench tab	Purpose
Workbench	Run POST /lab/analyze — upload image, inspect full response, view debug overlay
Gold Sets	Manage gold-set reference entries for benchmark evaluation
Candidates	Propose and track rule candidates through review lifecycle
Reference Dataset	Browse the 28-pattern reference library; ingest new images
Signals	Inspect session_signals hygiene — live/seeded/internal counts
Learning Ops	Trigger candidate auto-generation from accumulated signals
Benchmarks	Run benchmark suite, view pass/soft-pass/fail by pattern

Section 7 — Results Screen Cards

The results screen (`ui/src/screens/ResultsScreen.jsx`) assembles the analysis output into a sequence of cards. Cards are conditionally rendered based on whether a reference image was provided and the user's subscription tier. Phase labels organize cards into logical groups.

Card component	Always shown	Requires ref image	Description
LookSummaryCard	Yes	No	Pattern name, confidence label, key modifier and light count — free tier
BlueprintCard	Yes	No	Full blueprint YAML details — modifier, positions, distances, ratios
DiagramCard	Yes	No	SVG lighting diagram with clock-position annotations
ShootSetupCard	Yes	No	Plain-English setup instructions
SpaceCheckCard	Yes	No	Minimum room dimensions and ceiling requirements
CameraSubjectCard	Yes	No	Camera position, focal length, and subject framing guidance
HowToTestCard	Yes	No	Test-shot checklist for on-set verification
WhatToLookForCard	Yes	No	Visual confirmation cues when setup is correct
QuickFixesCard	Yes	No	Common failure modes and single-sentence corrections

RecommendedKitsCard	Yes	No	Gear recommendation cards matching the blueprint modifier family
OtherSetupsCard	Yes	No	Alternate patterns with similar aesthetic — jump to different result
SkinToneCard	Yes	No	Skin-tone specific power and modifier adjustment notes
SignalQualityCard	Yes	No	Classifier agreement, confidence tier, signal availability summary
TestShotCard	Yes	No	On-set test shot evaluation entry point — links to Shoot Mode
MySetupsCard	Yes	No	Save-to-kit and previously saved setups for this pattern
FeedbackCard	Yes	No	Outcome recording — nailed_it / close / failed — writes session_signal
ReferenceImageCard	No	Yes	The uploaded reference image with zoom overlay
RefImageReadCard	No	Yes	Technical read — hardware and modifier identification
RefLightingCard	No	Yes	Lighting analysis — direction, height, source, modifier
RefRecreationCard	No	Yes	Step-by-step recreation guide from Stage 7 reference read
RefInterpretationsCard	No	Yes	Alternative interpretations and confidence notes from the deep read

Section 8 — Debug Overlays

When `?debug=true` is passed to `POST /lab/analyze`, the engine generates a visual annotation overlay on top of the uploaded image. The overlay is saved to `/static/debug/_overlay.jpg` and its URL is returned as `debug_overlay_url` in the response.

What Overlays Annotate

Annotation	Color	Source signal
Face bounding box	Cyan	vision_data.face_box from MediaPipe
Shadow direction arrow	Amber	cue_report.primary_shadow_direction
Catchlight dots	White	vision_data.catchlights (per catchlight)
Highlight regions	Blue fill	cue_report.highlight_axis_map
Background region	Green outline	vision_data.region_attribution.background
Light position estimate	Orange dot	Derived from shadow + catchlight intersection
Pattern label	White text	authoritative_pattern + confidence_label

Accessing overlays

```
# Run lab analyze with debug flag
curl -X POST "http://localhost:8000/lab/analyze?debug=true" \
     -H "Authorization: Bearer <dev_jwt>" \
     -F "image=@reference.jpg"

# Response includes:
# "debug_overlay_url": "/static/debug/abc123_overlay.jpg"

# Fetch overlay:
curl http://localhost:8000/static/debug/abc123_overlay.jpg -o overlay.jpg
```

OVERLAY STORAGE

- Overlays are NOT automatically deleted — they accumulate in /static/debug/.
- Periodically clean up: rm -rf /static/debug/*.jpg (dev only).
- In production, overlays are behind dev auth — they are not publicly accessible.
- Overlay filenames are derived from the upload hash, not the original filename.

Section 9 — Visual Cue Report (24 Signals)

`VisualCueReport` in `engine/image_analysis_models.py` holds every signal the pipeline extracts from a single image. Each field is Optional — None means the signal could not be computed. `cue_report.cues_computed` tracks how many of the 24 were successfully extracted. Face-dependent signals return None when no face is detected.

Field	Face-dependent	What it measures
shadow_edge_hardness	Partial	Soft vs. hard shadow falloff — encodes source distance and diffusion
primary_shadow_direction	Yes	Clock position of key light derived from facial shadow geometry
vertical_light_angle	Yes	Height angle — high / eye-level / low — from shadow below brow/chin
catchlight_position	Yes	Clock position of catchlight in the iris
catchlight_shape	Yes	Round, softbox, ring, window, strip, or absent
catchlight_topology	Yes	Single / dual / ring / fill-only — counts and distribution
highlight_axis_map	No	Directional histogram of highlight regions across the frame
highlight_symmetry	No	Left/right highlight balance — encodes off-axis key strength
continuous_source_signals	No	Whether source appears continuous (LED panel) vs. flash

bounce_contributor	Partial	Probability and direction of a fill via bounce or reflector
separation_light	No	Presence and strength of rim / hair / separation light
off_axis_key	Yes	Degree to which key is placed off camera axis
light_structure	Yes	Direct pattern name derived from nose-shadow geometry
highlight_to_shadow_transition	Partial	Gradient width encodes feathering and modifier proximity
contrast_ratio	No	Highlight-to-shadow luminance ratio
subject_background_separation	No	Subject-background luminance and color separation
background_illumination	No	Background light pattern: even / gradient / spot / dark / natural
specular_highlight_behavior	No	Specular shape and distribution on skin/surfaces
reflection_architecture	No	Reflector-bounce geometry analysis
multi_shadow_detection	Partial	Multiple shadows indicating multi-light setup
environmental_shadow_continuity	No	Outdoor vs. studio shadow cues; foliage, window, mixed CT hints
pose_induced_shadow_interference	Yes	Body-pose shadows that could confuse direction detection
tonal_processing_estimation	No	B&W; detection; heavy contrast grading; tonal processing hints

shadow_interruption_pattern	Yes	Gobo / grid / projected pattern detected via shadow shape

Section 10 — Complete API Endpoint Reference

All 29 analysis-related endpoints across four route files. Auth column: Session = valid user JWT; Dev = JWT + email in NGW_DEV_EMAILS; None = unauthenticated.

Shoot-Match Routes

Method	Path	Auth	Purpose
POST	/shoot-match	Session	Full analysis or heuristic match — primary user endpoint
POST	/upload-reference	Session	Multipart image upload full pipeline analysis

Lab Analysis

Method	Path	Auth	Purpose
GET	/lab/status	Dev	Check dev access — returns email + NGW_DEV_MODE flag
POST	/lab/analyze	Dev	Full pipeline analysis + optional debug overlay

Gold Set

Method	Path	Auth	Purpose

GET	/lab/gold-set	Dev	List all gold-set entries
GET	/lab/gold-set/{entry_id}	Dev	Retrieve single entry by ID
POST	/lab/gold-set	Dev	Create new gold-set entry from analysis
PUT	/lab/gold-set/{entry_id}	Dev	Update entry (pattern, notes, flags)
DELETE	/lab/gold-set/{entry_id}	Dev	Remove entry from gold set
POST	/lab/gold-set/evaluate	Dev	Batch evaluate all gold-set entries against current engine

Rule Candidates

Method	Path	Auth	Purpose
GET	/lab/candidates	Dev	List all candidates
GET	/lab/candidates/{id}	Dev	Retrieve single candidate
POST	/lab/candidates	Dev	Create candidate from curator observation
PUT	/lab/candidates/{id}	Dev	Update status (proposed approved rejected)
DELETE	/lab/candidates/{id}	Dev	Remove candidate

Reference Library

Method	Path	Auth	Purpose
POST	/lab/reference-library/ingest	Dev	Ingest new reference image into library
POST	/lab/reference-library/rebuild-index	Dev	Rebuild search index after bulk changes
POST	/lab/reference-library/generate-legacy-sidecars	Dev	Generate sidecar JSON for legacy images
POST	/lab/reference-library/validate	Dev	Validate all library entries for schema compliance
GET	/lab/reference-library	Dev	List all library entries
GET	/lab/reference-library/{id}	Dev	Retrieve single entry metadata
POST	/lab/reference-library	Dev	Create entry (direct, no image processing)
PUT	/lab/reference-library/{id}	Dev	Update entry metadata
DELETE	/lab/reference-library/{id}	Dev	Remove from library
POST	/lab/reference-library/from-reconstruction	Dev	Create entry from VLM reconstruction output

Reference Dataset

Method	Path	Auth	Purpose

POST	/lab/reference-dataset/ingest	Dev	Ingest image into the 28-pattern reference dataset
GET	/lab/reference-dataset	Dev	List all dataset entries
GET	/lab/reference-dataset/version	Dev	Dataset version string
GET	/lab/reference-dataset/manifest	Dev	Full manifest with per-pattern counts
GET	/lab/reference-dataset/{pattern_id}/{ref_id}	Dev	Entry metadata
GET	/lab/reference-dataset/{pattern_id}/{ref_id}/image	Dev	Full-resolution image
GET	/lab/reference-dataset/{pattern_id}/{ref_id}/thumbnail	Dev	Thumbnail image
GET	/lab/reference-dataset/{pattern_id}/{ref_id}/debug-overlay	Dev	Debug overlay image
POST	/lab/reference-dataset/{pattern_id}/{ref_id}/approve	Dev	Approve entry for inclusion in training
POST	/lab/reference-dataset/{pattern_id}/{ref_id}/reject	Dev	Reject entry with reason
POST	/lab/reference-dataset/{pattern_id}/{ref_id}/reprocess	Dev	Re-run analysis on existing entry

Section 11 — Performance & Edge Cases

The full analysis pipeline processes a typical portrait image in 2–4 seconds on a production server (VLM latency dominates). Debug overlay generation adds 200–500 ms. The pipeline is designed to degrade gracefully — if any individual stage fails, `AnalysisResult.ok` remains True and the stage's output field is left at its safe default. Only a total pipeline failure sets `ok = False`.

Face Detection Edge Cases

Five of the 24 visual cues are face-dependent and return None when no face is detected: `primary_shadow_direction`, `vertical_light_angle`, `pose_induced_shadow_interference`, `shadow_interruption_pattern`, and `light_structure`. When face detection fails, the engine falls back to non-face cues (highlight axis map, background illumination, environmental shadows) and sets `edge_case_flags.no_face = True` and `face_validation.face_quality = "none"`.

FaceValidation Fields

Field	Type	Description
face_detected	bool	True when MediaPipe face detection returns a bounding box
face_confidence	float	MediaPipe detection confidence score 0.0–1.0
face_quality	str	"good" (large area, yaw present) | "partial" | "none"
face_yaw	float|None	Face yaw angle in degrees; None if yaw cannot be computed

face_box_area_ratio	float	face_area / image_area — small ratio (<0.01) = "tiny_face" flag

EdgeCaseFlags

Flag	Trigger condition
no_face	face_validation.face_detected is False
blown_highlights	contrast_ratio.ratio > 8.0
mixed_color_temperature	Both warm_* and cool_* hints in environmental_shadow_continuity
outdoor_foliage_shadows	"dappled_foliage" in environmental_shadow_continuity.environment_hints
window_light_gradient	background_illumination.pattern == "gradient" AND natural indicators
extreme_low_key	classification.brightness == "low" AND light_structure.shadow_density > 0.5
bw_processing	tonal_processing_estimation.is_bw is True

SignalReliability

Populated by the perception layer after all stages complete. `signals_available` counts non-None cue fields on `VisualCueReport`. `weak_signals` lists cue names whose confidence attribute is below 0.3. `overall_signal_strength` is computed from `cue_report.overall_confidence()`. When `signals_available < 10`, the `"low_signal_count"` ambiguity flag is set in `perception_explanation.ambiguity_flags`.